

# Subtask 1 - Encoder-Only Transformers for Relational Reasoning

## Overview

In this task, you will explore how encoder-only transformers process and reason over structured numerical data. You will build and analyze a BERT-style architecture for solving an **Array Element Ranking** problem.

Unlike autoregressive models that process tokens sequentially, encoder-only transformers view the entire sequence simultaneously using bidirectional self-attention. This makes them particularly effective for tasks requiring global relational understanding.

Through this assignment, you will investigate:

- how self-attention performs implicit pairwise comparisons across a sequence,
- how numerical information can be represented inside transformer architectures,
- how positional encodings affect relational reasoning,
- how attention differs from recurrence for global context tasks,
- and how transformers behave under out-of-distribution inputs.

The primary objective is not only to obtain good performance, but also to understand *why* the model behaves the way it does.

---

## Problem Statement

You are given a sequence of integers:

[45, 12, 99, 31]

The task is to predict the **relative sorted rank** of each element.

Sorted sequence:

[12, 31, 45, 99]

Corresponding ranks:

[2, 0, 3, 1]

Meaning:

- 12 is the smallest → rank 0
- 31 is second smallest → rank 1
- 45 is third smallest → rank 2
- 99 is largest → rank 3

The model must output the rank of every token in the input sequence.

This is fundamentally a **global relational reasoning task**, since determining the rank of a number requires comparing it against all other numbers in the sequence.

---

## Dataset

You will work with a synthetic dataset available [here](#).

---

## Suggested Workflow

The following workflow is only a guideline. You are encouraged to go beyond it.

---

### 1. Build Baselines

Before implementing transformers, start with simpler architectures to understand why ranking is difficult for traditional sequence models.

Possible baselines include:

- Multi-Layer Perceptrons (MLPs)
- RNNs
- LSTMs

### Suggested Questions

- How quickly do these models converge?
- Do sequential models struggle with global comparisons?
- Does performance degrade with longer sequences?
- Can the models generalize to unseen patterns?

## Compare

- Training convergence
  - Validation accuracy
  - Stability
  - Generalization behavior
  - Computational efficiency
- 

## 2. Implement an Encoder-Only Transformer

Implement and train a transformer encoder from scratch for the ranking task.

Treat the problem as a **token classification problem**, where each token predicts its own rank.

Your implementation should include:

- Bidirectional self-attention
- Positional encodings
- Multi-head attention
- Feed-forward networks
- Residual connections
- Layer normalization
- Classification head for rank prediction

## Important Note

You may use low-level PyTorch or TensorFlow primitives.

However, the attention mechanism and transformer blocks should ideally be implemented manually instead of importing high-level transformer libraries. The goal is to understand:

- tensor shapes,
- Q/K/V projections,
- attention score computation,

- masking behavior,
  - and multi-head aggregation.
- 

## 3. Inference, Visualization & Analysis

Transformers are highly sensitive to data representation.

After training:

### Attention Visualization

Extract and visualize attention weights using heatmaps.

Analyze questions such as:

- How does a token attend to larger or smaller numbers?
- Does the model compare elements pairwise?
- Do different heads specialize differently?

Example:

- Does token 45 strongly attend to token 99 while determining its rank?
  - Does attention become more structured after training?
- 

### Out-of-Distribution Testing

Evaluate the model on sequences that differ significantly from the training distribution.

Examples:

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

[1000, 900, 800, 700, ...]

[5, 5, 5, 5, 5]

Analyze:

- robustness,
- failure modes,

- attention behavior,
  - and generalization capability.
- 

# Ablations & Experiments

A major component of this assignment is experimentation and analysis.

You are expected to conduct meaningful ablation studies regarding how transformers process non-linguistic numerical data.

---

## A. Data Representation

This is one of the most important aspects of the assignment.

### Categorical Embeddings vs Continuous Representations

Investigate:

- embedding layers,
- learned projections,
- normalized numerical inputs,
- raw float inputs.

Questions to explore:

- Does the model treat 5 and 6 as mathematically related?
  - Or does it treat them as unrelated symbols?
- 

### Sequence Normalization

Experiment with:

- Z-score normalization,
- min-max scaling,
- sequence-wise normalization.

Analyze how normalization affects:

- training stability,
  - convergence,
  - out-of-distribution performance.
- 

## **B. Architecture & Attention**

### **RNN/LSTM vs Transformer**

Compare:

- sequential processing,
  - long-range reasoning,
  - parallelism,
  - global context understanding.
- 

### **Positional Encoding Ablation**

Remove positional encodings entirely.

Questions:

- Can the model still rank numbers?
  - Does order matter for this task?
  - What information is lost?
- 

### **Attention Head Specialization**

If using multiple attention heads:

- visualize all heads,
- compare their patterns,
- analyze specialization.

Do some heads focus on:

- large numbers?
  - nearby tokens?
  - pairwise comparisons?
- 

## Depth Experiments

Compare:

- 1-layer encoders
- 2-layer encoders
- 4-layer encoders

Analyze:

- convergence,
  - expressiveness,
  - overfitting,
  - attention structure.
- 

## Deliverables

### 1. Source Code

Submit:

- dataset generation scripts,
- baseline implementations,
- transformer implementation,
- training scripts,
- evaluation/inference code.

Your code should be:

- clean,
  - reproducible,
  - properly structured,
  - and easy to run.
-

## 2. Experimental Report

Submit a short report containing:

- methodology,
- architectural choices,
- numerical representation strategies,
- experiments performed,
- ablation studies,
- evaluation metrics,
- attention visualizations,
- conclusions and observations.

Focus on analysis and reasoning, not just metrics.

---

## 3. Trained Model + Demo

Submit (mandatorily):

- trained checkpoints,
- inference notebook.

### Suggested Demo Ideas

- User inputs 10 numbers
- Model predicts ranks
- Attention heatmap is visualized alongside predictions

Optional:

- interactive UI,
  - web demo,
  - visualization dashboard.
- 

## Suggested Metrics

You may use:



## Training Metrics

- Cross-Entropy Loss
- Validation Loss

## Evaluation Metrics

- Token-level Accuracy
  - Exact Sequence Accuracy
  - Out-of-Distribution Accuracy
- 

## Recommended Focus Areas

The most important aspect of this assignment is understanding:

- how transformers reason globally,
- how attention performs comparisons,
- how representation affects learning,
- and how architectural choices influence behavior.

Visualization and interpretation are strongly encouraged.

Do not treat this as only a benchmarking task.

---

## Expected Outcomes

By the end of this assignment, you should be comfortable with:

- implementing transformer encoders from scratch,
- understanding attention mathematically,
- representing numerical data in transformers,
- analyzing attention patterns,
- designing ablation studies,
- and evaluating model generalization behavior.